

形式化方法 大作业实验1 死锁检验

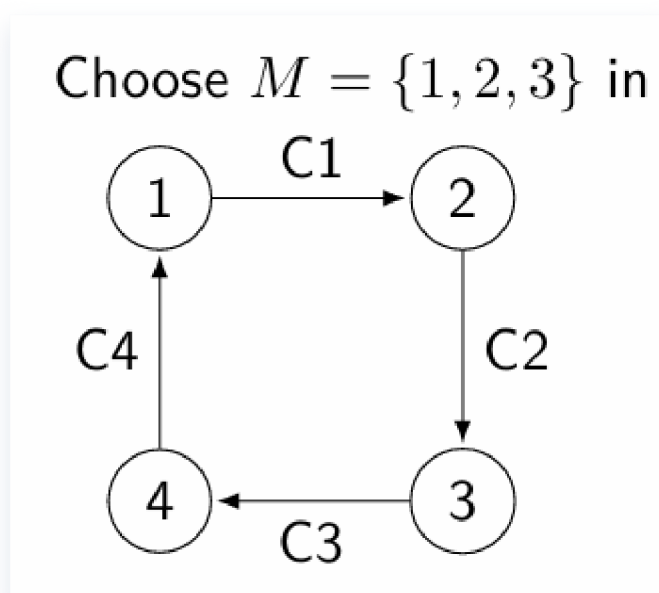
PB22111599 杨映川

1 实验内容

根据PPT内容构建死锁检验，分别对一个简单模型（4个节点）和一个复杂模型（17个节点）进行死锁检验。

2 简单模型

2.1 建模分析



1. 只有M中的节点可以收发信息；
2. M中的节点可以进行
 1. send动作，向一个空信道发送信息；
 2. process动作，将一个信息转发到下一个空的信道，并清零当前信道；
 3. receive动作，当信息来到指向其目标的信道时，接收成功，并将信道重新置为0；

3. 由于M中的节点可以send或者receive, 这个选择将完全随机;

回顾: write $OK(n, m, c)$, if it is allowed to pass the message to the outgoing channel c from n , when a message has to be sent from n to m

- *send* steps: if $OK(n, m, c)$, then



- *receive* steps:



- *processing* steps: if $OK(n, m, c')$, then



2.2 代码实现

参见文件 `easy.smv`

1. 单独定义了node这个模块, 对每个节点进行单独的管理。

```

1  MODULE main
2      VAR
3          c1: 0..4;
4          c2: 0..4;
5          c3: 0..4;
6          c4: 0..4;
7          -- 不允许给4发送消息
8          -- M = {1, 2, 3}
9          -- c_in, id, c_out, 可发的信息, send权限
10         pr1: process node(c4, 1, c1, {2, 3}, TRUE);
11         pr2: process node(c1, 2, c2, {1, 3}, TRUE);
12         pr3: process node(c2, 3, c3, {2, 1}, TRUE);
13         pr4: process node(c3, 4, c4, {2, 3, 1}, FALSE);
14     ASSIGN
15         -- 初始化所有信道为空
16         init(c1) := 0;
17         init(c2) := 0;
18         init(c3) := 0;
19         init(c4) := 0;
20         next(c1) := c1;
21         next(c2) := c2;
22         next(c3) := c3;
23         next(c4) := c4;
24     CTLSPEC
25         -- 在所有情况下都不可能所有信道均不为空,
26         -- 并且所有节点都无法receive信息
27         AG(!(c1≠0&c1≠2&c2≠0&c2≠3&c3≠0&c4≠0&c4≠1))

```

```

28
29 MODULE node(c_in, id, c_out, mlist, allow)
30     FAIRNESS running
31     VAR
32         st : {send, proc, recv};
33     ASSIGN
34         -- 一个节点状态可以是send或者process或者receive
35         init(st) := {send, proc, recv};
36         next(st) := {send, proc, recv};
37         next(c_in) :=
38         case
39             -- in信道上是自己, 可以接收, 然后置空
40             (c_in = id & st = recv) : 0;
41             -- 如果当前状态为process状态, 并且有信息可以转发, 则进行转
发, 将c_in设为空
42             (c_in ≠ 0 & c_in ≠ id & c_out = 0 & st = proc) : 0;
43             -- 否则保持c_in不变
44             TRUE : c_in;
45         esac;
46         next(c_out) :=
47         case
48             -- 如果当前状态为send, 则随机向一个node发送一条信息
49             (c_out = 0 & allow = TRUE & st = send) : mlist;
50             -- 如果当前状态为process状态, 并且有信息可以转发, 则让c_out
= c_in
51             (c_in ≠ 0 & c_in ≠ id & c_out = 0 & st = proc) :
c_in;
52             -- 否则保持c_out不变
53             TRUE : c_out;
54         esac;

```

2.3 检验结果

运行 NuSMV easy.smv , 得到如下结果

```

1  -- specification AG !((((c1 ≠ 0 & c1 ≠ 2) & c2 ≠ 0) & c2 ≠
2  3) & c3 ≠ 0) & c4 ≠ 0) & c4 ≠ 1) is false
3  -- as demonstrated by the following execution sequence
4  Trace Description: CTL Counterexample
5  Trace Type: Counterexample
6  → State: 1.1 ←
7      c1 = 0
8      c2 = 0
9      c3 = 0
10     c4 = 0
11     pr1.st = send
12     pr2.st = send
13     pr3.st = send
14     pr4.st = send
15     → Input: 1.2 ←

```

```

15     _process_selector_ = pr1
16     running = FALSE
17     pr4.running = FALSE
18     pr3.running = FALSE
19     pr2.running = FALSE
20     pr1.running = TRUE
21 → State: 1.2 ←
22     c1 = 3
23 → Input: 1.3 ←
24     _process_selector_ = pr2
25     pr2.running = TRUE
26     pr1.running = FALSE
27 → State: 1.3 ←
28     c2 = 1
29 → Input: 1.4 ←
30     _process_selector_ = pr3
31     pr3.running = TRUE
32     pr2.running = FALSE
33 → State: 1.4 ←
34     c3 = 2
35 → Input: 1.5 ←
36     _process_selector_ = pr4
37     pr4.running = TRUE
38     pr3.running = FALSE
39 → State: 1.5 ←
40     pr4.st = proc
41 → Input: 1.6 ←
42 → State: 1.6 ←
43     c3 = 0
44     c4 = 2
45     pr4.st = send
46 → Input: 1.7 ←
47     _process_selector_ = pr3
48     pr4.running = FALSE
49     pr3.running = TRUE
50 → State: 1.7 ←
51     c3 = 1

```

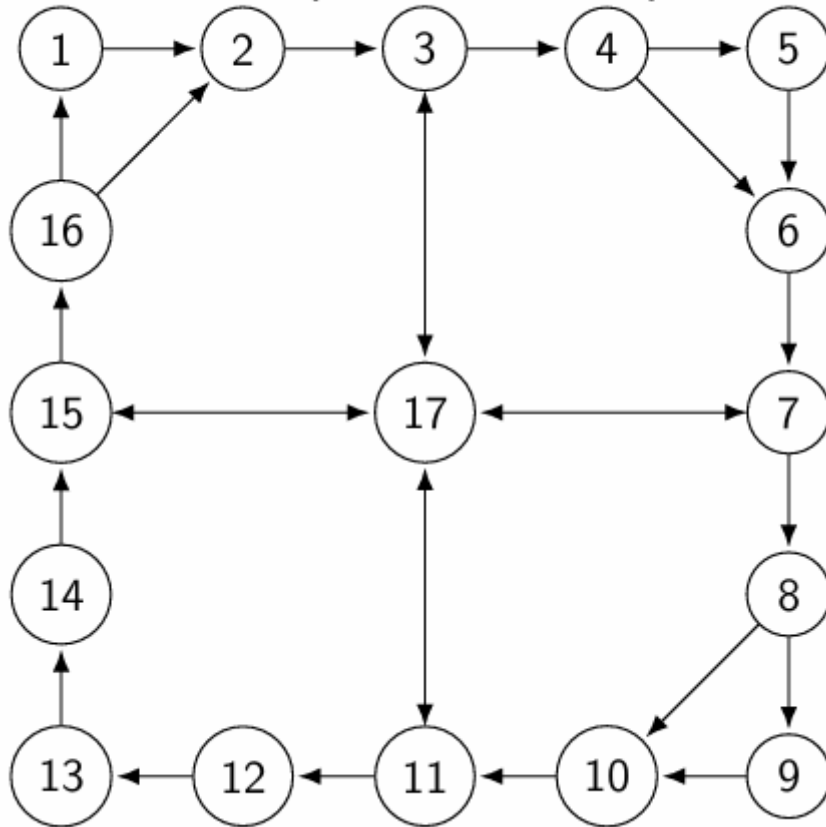
可以看到最终有 $c1 = 3, c2 = 1, c3 = 1, c4 = 2$, 和PPT上的示例一样, 此时陷入死锁。

3 复杂模型其一 $M=\{2, 4, 6\}$

3.1 模型分析与简化

接下来我们把检验方法推广到复杂模型

A more complicated example:



该例子有23个channel，而我们当前的主节点只有三个（2,4,6），为了防止耗费过多时间检验不必要的状态转移，先根据M中节点相互到达的最短路径对模型进行“剪枝”

- 2-4: 2-3-4
- 2-6: 2-3-4-6
- 4-2: 4-6-7-17-15-16-2
- 4-6: 4-6
- 6-2: 6-7-17-15-16-2
- 6-4: 6-7-17-3-4

因此保留节点**2,3,4,6,7,15,16,17**，其他节点可以删掉且不影响我们对模型的检验。

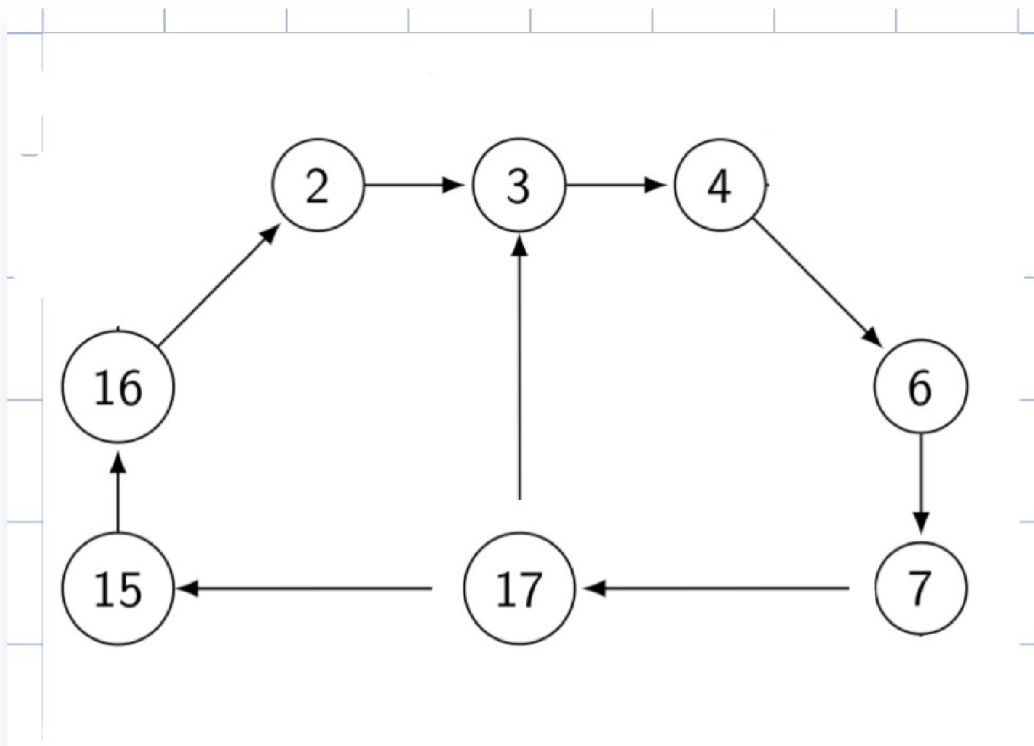
另外，只需要考虑17->3而不需要考虑反过来，17->15和7->17同理。

也就是说，我们不仅去掉了无关的节点，还去掉了无关的信道。

为什么？

- 因为在检验死锁时，只需要考虑M中的节点在收发消息时的所有环路，没有参与到最短路径中的节点实际上是对路径选择的一个扩充，本质上可以被归约到最短路径上。

此时，模型图简化为如下,大幅缩小了状态空间。



接下来可以将节点分成类

- 一进一出可收发：2,4,6
- 一进一出不可收发：7,15,16
- 二进一出不可收发：3
- 一进二出不可收发：17

3.2 代码实现

参见文件 `hard2.smv`

3.2.1 初始化

定义能传递的信息列表，以及定义不同类型的节点、信道连接关系

```

1  MODULE main
2      VAR
3      -- 定义每个channel可以传送的信息范围
4          c_2_3: {0, 2, 4, 6};
5          c_3_4: {0, 2, 4, 6};
6          c_4_6: {0, 2, 4, 6};
7          c_6_7: {0, 2, 4, 6};
8          c_7_17: {0, 2, 4, 6};
9          c_17_3: {0, 2, 4, 6};
10         c_17_15: {0, 2, 4, 6};
11         c_15_16: {0, 2, 4, 6};
12         c_16_2: {0, 2, 4, 6};
13     -- 定义每个process转发
14     -- 一进一出可收发
15     pr2: process node1ly(c_16_2, 2, c_2_3, {4, 6});
16     pr4: process node1ly(c_3_4, 4, c_4_6, {2, 6});

```

```

17     pr6: process node11y(c_4_6, 6, c_6_7, {2, 4});
18     -- 一进一出不可收发
19     pr7: process node11n(c_6_7, 7, c_7_17);
20     pr15: process node11n(c_17_15, 15, c_15_16);
21     pr16: process node11n(c_15_16, 16, c_16_2);
22     -- 二进一出不可收发
23     pr3: process node21n(c_2_3, c_17_3, 3, c_3_4);
24     -- 一进二出不可收发
25     pr17: process node12n(c_7_17, 17, c_17_15, c_17_3);
26     ASSIGN
27         -- 初始化每个channel为0(empty)
28         init(c_2_3) := 0;
29         init(c_3_4) := 0;
30         init(c_4_6) := 0;
31         init(c_6_7) := 0;
32         init(c_7_17) := 0;
33         init(c_17_3) := 0;
34         init(c_17_15) := 0;
35         init(c_15_16) := 0;
36         init(c_16_2) := 0;
37         next(c_2_3) := c_2_3;
38         next(c_3_4) := c_3_4;
39         next(c_4_6) := c_4_6;
40         next(c_6_7) := c_6_7;
41         next(c_7_17) := c_7_17;
42         next(c_17_3) := c_17_3;
43         next(c_17_15) := c_17_15;
44         next(c_15_16) := c_15_16;
45         next(c_16_2) := c_16_2;

```

3.2.2 对于阻塞的情况

- 若是收发节点，当前入边不是正确消息且出边被占用；
- 若是中转节点，
 - 当前入边有一条消息要传递，但是这条消息所需的出边被占用；
 - 出入边均为空（因为当前用不到）；

什么是“这条消息所需的出边”被占用？

- 指的是该条消息所需要走的那一条最短路径被阻塞了

```

1     CTLSPEC
2         AG(! (
3             -- 可收发节点(2,4,6)阻塞: 收不到正确消息且发送通道被占用
4             (c_2_3 ≠ 0 & c_16_2 ≠ 2) &
5             (c_4_6 ≠ 0 & c_3_4 ≠ 4) &

```

```

6          (c_6_7 ≠ 0 & c_4_6 ≠ 6) &
7
8          -- 一进一出不可收发节点(7,15,16)阻塞: 要么入出通道都非空, 要
么都为空
9          ((c_6_7 ≠ 0 & c_7_17 ≠ 0) | (c_6_7 = 0 & c_7_17 =
0)) &
10         ((c_17_15 ≠ 0 & c_15_16 ≠ 0) | (c_17_15 = 0 &
c_15_16 = 0)) &
11         ((c_15_16 ≠ 0 & c_16_2 ≠ 0) | (c_15_16 = 0 & c_16_2
= 0)) &
12
13         -- 二进一出不可收发节点(3)阻塞: 要么出边被占用且至少一个入边有
消息, 要么所有通道都空
14         ((c_3_4 ≠ 0 & (c_2_3 ≠ 0 | c_17_3 ≠ 0)) | (c_2_3 =
0 & c_17_3 = 0 & c_3_4 = 0)) &
15
16         -- 一进二出不可收发节点(17)阻塞: 要么入边有消息且其所需出边被
占用, 要么所有通道都空
17         (((c_7_17 = 2 & c_17_15 ≠ 0) | (c_7_17 = 4 & c_17_3
≠ 0)) | (c_7_17 = 0 & c_17_15 = 0 & c_17_3 = 0))
18         ))
19

```

3.2.3 接下来逐一各类节点

1. 对于可收发的一进一出节点, 类似easy模型的定义方式即可
2. 对于不可收发的一进一出节点, 仅考虑其发挥中转作用

```

1  MODULE node11y(c_in, id, c_out, mlist)
2      FAIRNESS running
3      VAR
4          st : {send, proc, recv};
5      ASSIGN
6          init(st) := {send, proc, recv};
7          next(st) := {send, proc, recv};
8          next(c_in) :=
9      case
10         -- receive
11         (c_in = id & st = recv) : 0;
12         -- process
13         (c_in ≠ 0 & c_in ≠ id & c_out = 0 & st = proc) : 0;
14         -- idle
15         TRUE : c_in;
16     esac;
17     next(c_out) :=
18     case
19         -- send
20         (c_out = 0 & st = send) : mlist;
21         -- process
22         (c_in ≠ 0 & c_in ≠ id & c_out = 0 & st = proc) :
c_in;

```



```

23         -- idle
24         TRUE : c_out;
25     esac;
26
27     MODULE node11n(c_in, id, c_out)
28         FAIRNESS running
29         ASSIGN
30             next(c_in) :=
31             case
32                 -- process only
33                 (c_in ≠ 0 & c_out = 0) : 0;
34                 -- idle
35                 TRUE : c_in;
36             esac;
37             next(c_out) :=
38             case
39                 -- process only
40                 (c_in ≠ 0 & c_out = 0) : c_in;
41                 -- idle
42                 TRUE : c_out;
43             esac;

```

3. 对于不可收发的二进一出节点即3号节点，首先看出

1. 当2->4或2->6时，要选取2-3-4这条路径，可能转发4或6
2. 当6->4时，要选取17-3-4这条路径，可能转发4

此时为了能够使代码能够判断具体选择了那一条路径，我们引入一个新的变量 `active_channel` 来进行标识，避免出现两个入边因存在相同的消息4而被同时置为零的情况。

```

1  MODULE node21n(c_in1, c_in2, id, c_out)
2      FAIRNESS running
3      VAR
4          -- 添加状态变量跟踪正在处理的通道
5          active_channel: {none, from_c_in1, from_c_in2};
6
7      ASSIGN
8          init(active_channel) := none;
9
10         -- 控制当前处理哪个通道
11         next(active_channel) :=
12         case
13             -- 如果输出通道空闲，选择一个有效消息的输入通道处理
14             active_channel = none & c_out = 0 & (c_in1 = 4 |
c_in1 = 6): from_c_in1;
15             active_channel = none & c_out = 0 & c_in2 = 4:
from_c_in2;
16             -- 处理完后重置状态
17             active_channel ≠ none & c_out ≠ 0: none;
18             TRUE: active_channel;

```

```

19         esac;
20
21         -- c_in1的状态转移规则
22         next(c_in1) :=
23         case
24             -- 只有当选择处理c_in1时才清空它
25             active_channel = from_c_in1 & (c_in1 = 4 | c_in1 = 6)
& c_out = 0: 0;
26             TRUE: c_in1;
27         esac;
28
29         -- c_in2的状态转移规则
30         next(c_in2) :=
31         case
32             -- 只有当选择处理c_in2时才清空它
33             active_channel = from_c_in2 & c_in2 = 4 & c_out = 0:
0;
34             TRUE: c_in2;
35         esac;
36
37         -- c_out的状态转移规则
38         next(c_out) :=
39         case
40             -- 根据选择的通道设置输出
41             active_channel = from_c_in1 & (c_in1 = 4 | c_in1 = 6)
& c_out = 0: c_in1;
42             active_channel = from_c_in2 & c_in2 = 4 & c_out = 0:
c_in2;
43             TRUE: c_out;
44         esac;

```

4. 对于不可收发的一进二出的节点17，同样的分析方法得到

1. 对于4-→2或6-→2，选择路径7-17-15，可能转发2

2. 对于6-→4，选择路径7-17-3，可能转发4

此时不会冲突，可以直接按照入边消息具体的值进行迭代

```

1  MODULE node12n(c_in, id, c_out1, c_out2)
2      FAIRNESS running
3      ASSIGN
4          next(c_in) :=
5          case
6              -- 4→2或6→2
7              (c_in = 2 & c_out1 = 0) : 0;
8              -- 6→4
9              (c_in = 4 & c_out2 = 0) : 0;
10             -- idle
11             TRUE : c_in;
12         esac;
13         next(c_out1) :=

```

```

14         case
15             -- process only
16             (c_in = 2 & c_out1 = 0) : c_in;
17             -- idle
18             TRUE : c_out1;
19         esac;
20         next(c_out2) :=
21         case
22             -- process only
23             (c_in = 4 & c_out2 = 0) : c_in;
24             -- idle
25             TRUE : c_out2;
26         esac;

```

3.3 检验结果

运行 NuSMV `hard2.smv` , 得到如下结果

```

1  -- specification AG !(((((((c_2_3 ≠ 0 & c_16_2 ≠ 2) & (c_4_6
   ≠ 0 & c_3_4 ≠ 4)) & (c_6_7 ≠ 0 & c_4_6 ≠ 6)) & ((c_6_7 ≠ 0 &
   c_7_17 ≠ 0) | (c_6_7 = 0 & c_7_17 = 0))) & ((c_17_15 ≠ 0 &
   c_15_16 ≠ 0) | (c_17_15 = 0 & c_15_16 = 0))) & ((c_15_16 ≠ 0 &
   c_16_2 ≠ 0) | (c_15_16 = 0 & c_16_2 = 0))) & ((c_3_4 ≠ 0 &
   (c_2_3 ≠ 0 | c_17_3 ≠ 0)) | ((c_2_3 = 0 & c_17_3 = 0) & c_3_4 =
   0))) & (((c_7_17 = 2 & c_17_15 ≠ 0) | (c_7_17 = 4 & c_17_3 ≠
   0)) | ((c_7_17 = 0 & c_17_15 = 0) & c_17_3 = 0))) is false
2  -- as demonstrated by the following execution sequence
3  Trace Description: CTL Counterexample
4  Trace Type: Counterexample
5  → State: 1.1 ←
6      c_2_3 = 0
7      c_3_4 = 0
8      c_4_6 = 0
9      c_6_7 = 0
10     c_7_17 = 0
11     c_17_3 = 0
12     c_17_15 = 0
13     c_15_16 = 0
14     c_16_2 = 0
15     pr2.st = send
16     pr4.st = send
17     pr6.st = send
18     pr3.active_channel = none
19  → Input: 1.2 ←
20     _process_selector_ = pr2
21     running = FALSE
22     pr17.running = FALSE
23     pr3.running = FALSE
24     pr16.running = FALSE
25     pr15.running = FALSE

```

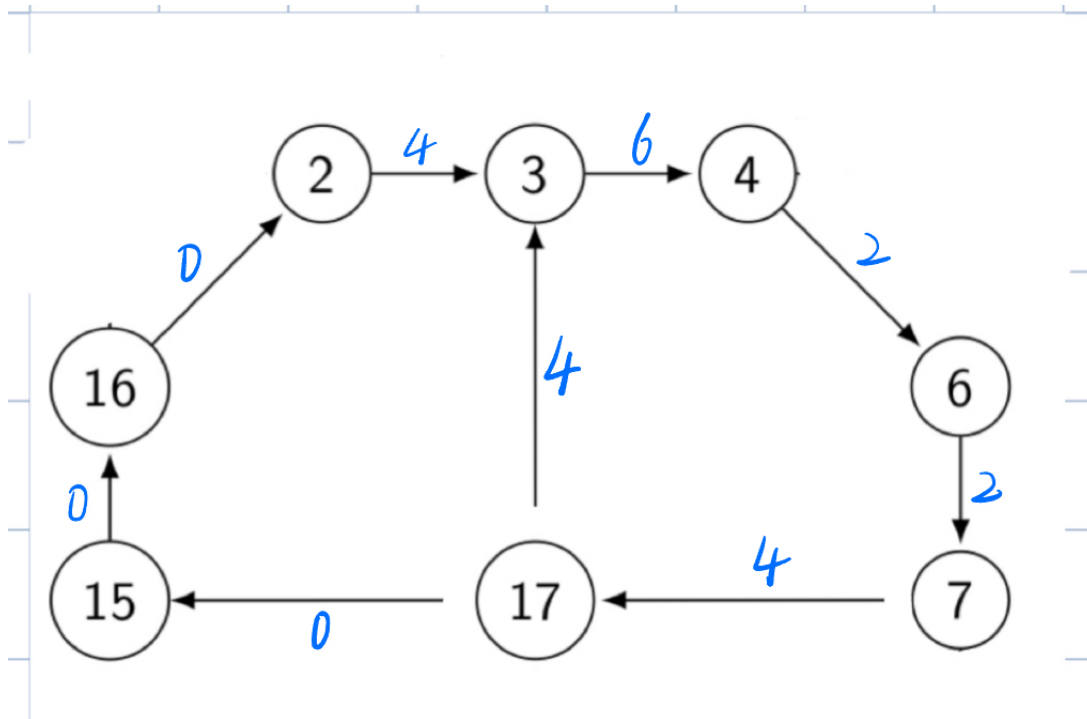
```
26     pr7.running = FALSE
27     pr6.running = FALSE
28     pr4.running = FALSE
29     pr2.running = TRUE
30 → State: 1.2 ←
31     c_2_3 = 6
32 → Input: 1.3 ←
33     _process_selector_ = pr4
34     pr4.running = TRUE
35     pr2.running = FALSE
36 → State: 1.3 ←
37     c_4_6 = 2
38 → Input: 1.4 ←
39     _process_selector_ = pr6
40     pr6.running = TRUE
41     pr4.running = FALSE
42 → State: 1.4 ←
43     c_6_7 = 4
44 → Input: 1.5 ←
45     _process_selector_ = pr7
46     pr7.running = TRUE
47     pr6.running = FALSE
48 → State: 1.5 ←
49     c_6_7 = 0
50     c_7_17 = 4
51 → Input: 1.6 ←
52     _process_selector_ = pr6
53     pr7.running = FALSE
54     pr6.running = TRUE
55 → State: 1.6 ←
56     c_6_7 = 4
57 → Input: 1.7 ←
58     _process_selector_ = pr17
59     pr17.running = TRUE
60     pr6.running = FALSE
61 → State: 1.7 ←
62     c_7_17 = 0
63     c_17_3 = 4
64 → Input: 1.8 ←
65     _process_selector_ = pr7
66     pr17.running = FALSE
67     pr7.running = TRUE
68 → State: 1.8 ←
69     c_6_7 = 0
70     c_7_17 = 4
71 → Input: 1.9 ←
72     _process_selector_ = pr6
73     pr7.running = FALSE
74     pr6.running = TRUE
75 → State: 1.9 ←
76     c_6_7 = 2
77 → Input: 1.10 ←
```

```

78     _process_selector_ = pr3
79     pr3.running = TRUE
80     pr6.running = FALSE
81     → State: 1.10 ←
82     pr3.active_channel = from_c_in1
83     → Input: 1.11 ←
84     → State: 1.11 ←
85     c_2_3 = 0
86     c_3_4 = 6
87     → Input: 1.12 ←
88     _process_selector_ = pr2
89     pr3.running = FALSE
90     pr2.running = TRUE
91     → State: 1.12 ←
92     c_2_3 = 4

```

整理得到，当出现以下情形时触发死锁（蓝色表示当前该信道上的消息）



4 复杂模型其二 $M=\{1, 5, 9, 13\}$

4.1 模型分析与简化

同上一个模型，进行同样的分析和模型简化

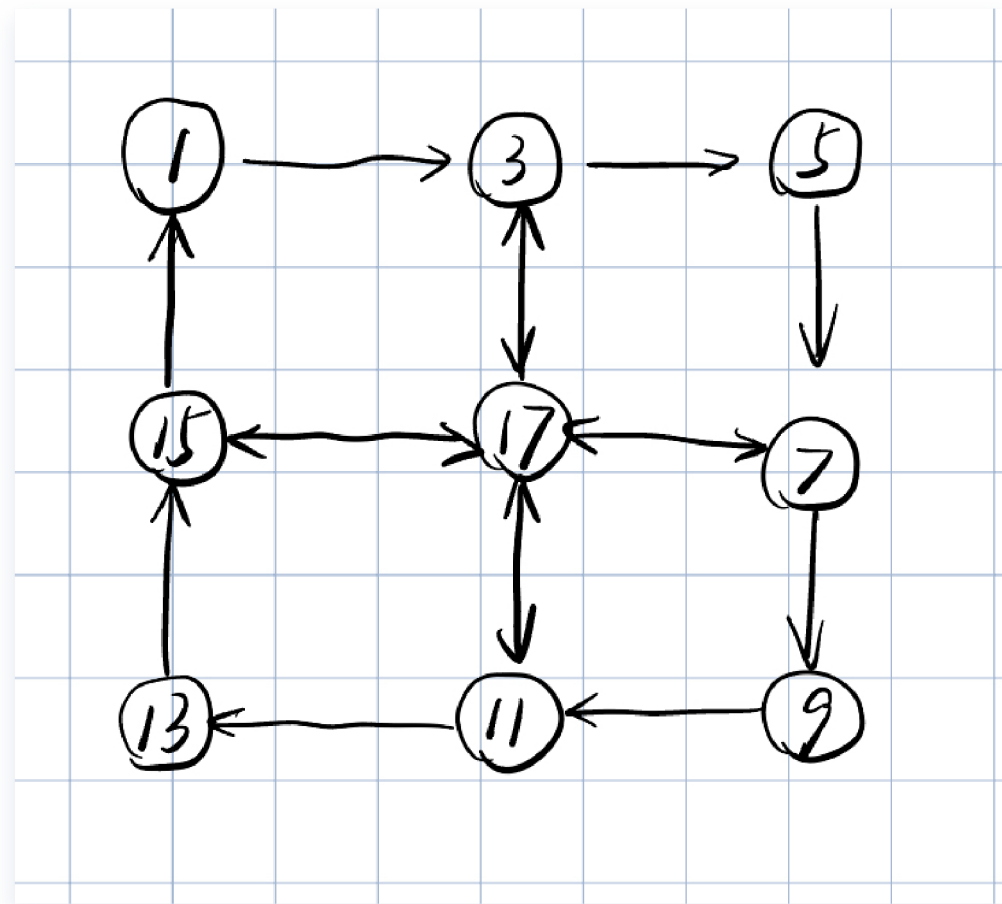
先列出相互可达的最短路径，取出必要的节点和信道

- 1->5: 1-2-3-4-5
- 1->9: 1-2-3-17-7-8-9
- 1->13: 1-2-3-17-11-12-13
- 5->9: 5-6-7-8-9
- 5->13: 5-6-7-17-11-12-13
- 5->1: 5-6-7-17-15-16-1

- 9->13: 9-10-11-12-13
- 9->1: 9-10-11-17-15-16-1
- 9->5: 9-10-11-17-3-4-5
- 13->1: 13-14-15-16-1
- 13->5: 13-14-15-17-3-4-5
- 13->9: 13-14-15-17-7-8-9

1. 首先所有节点都出现了，无法删除单个节点；
2. 我们观察到 16->2, 4->6, 8->10 这三条边从未使用，删去；
3. 我们观察到一些节点转移总是一起出现：2总是在1-3之间；4总是在3-5之间；6总是在5-7之间；8总是在7-9之间；10总是在9-11之间；12总是在11-13之间；14总是在13-15之间；16总是在15-1之间；这些节点没有存在的必要，全部归约成一条信道。

如此一来我们得到一个简化的模型如下：



有三类节点

- 一进一出可收发：1,5,9,13
- 一进一出加一条双向边，不可收发：3,7,11,15
- 四条双向边不可收发：17

开始写代码！

4.2 代码实现

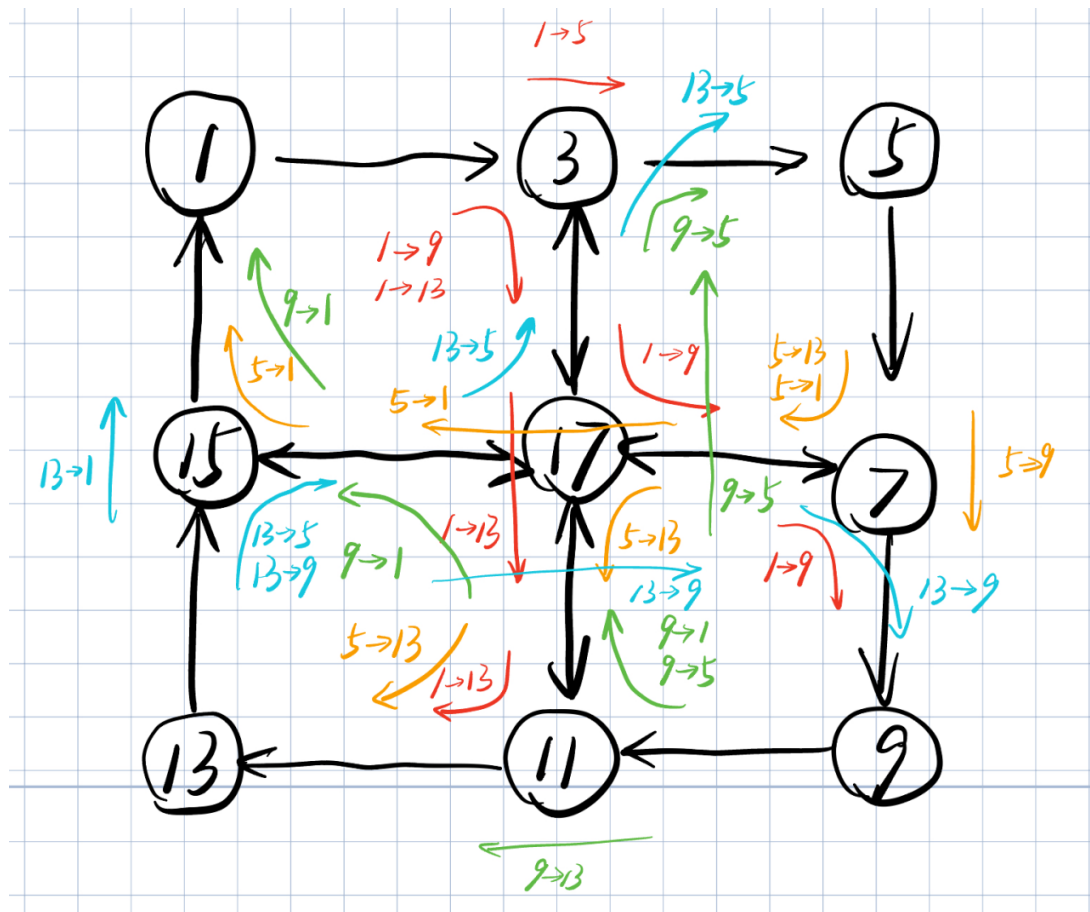
4.2.1 初始化

不再赘述，定义方法与之前一样

4.2.2 死锁检验

- 若是收发节点，当前入边不是正确消息且出边被占用；
- 若是中转节点，
 - 当前入边有一条消息要传递，但是这条消息所需的出边被占用；
 - 出入边均为空（因为当前用不到）；

由于我们简化了模型，在判断死锁条件之前需要对特定路线上可流通的信息选项进行分析



- 对于节点3，
 - 1-3-5: 1 to 5, 流通5。当c_1_3有5时，c_3_5非空，达成阻塞条件；
 - 1-3-17: 1 to 9 or 1 to 13, 流通9,13。同上理
 - 17-3-5: 13 to 5 or 9 to 5, 流通5。同上理
- 其他节点的分析类似，可以参考上图

```
1 CTLSPEC
2     AG(! (
3         -- 可收发节点(1,5,9,13)阻塞: 收不到正确消息且发送通道被占用
4         (c_15_1 ≠ 1 & c_1_3 ≠ 0) &
5         (c_3_5 ≠ 5 & c_5_7 ≠ 0) &
6         (c_7_9 ≠ 9 & c_9_11 ≠ 0) &
7         (c_11_13 ≠ 13 & c_13_15 ≠ 0) &
```

```

8
9      -- 二进二出不可收发节点(3,7,11,15)阻塞
10     -- 节点3阻塞: 入边有消息但所需出边被占用, 或所有通道为空
11     ((c_1_3 = 5 & c_3_5 ≠ 0) | (c_1_3 = 9 & cnorth ≠ 0)
12 | (c_1_3 = 13 & cnorth ≠ 0) |
13     (cnorth = 5 & c_3_5 ≠ 0) | (c_1_3 = 0 & cnorth = 0
14 & c_3_5 = 0)) &
15
16     -- 节点7阻塞
17     ((c_5_7 = 9 & c_7_9 ≠ 0) | (c_5_7 = 1 & ceast ≠ 0)
18 | (c_5_7 = 13 & ceast ≠ 0) |
19     (ceast = 9 & c_7_9 ≠ 0) | (c_5_7 = 0 & ceast = 0 &
20 c_7_9 = 0)) &
21
22     -- 节点11阻塞
23     ((c_9_11 = 13 & c_11_13 ≠ 0) | (c_9_11 = 1 & csouth
24 ≠ 0) | (c_9_11 = 5 & csouth ≠ 0) |
25     (csouth = 13 & c_11_13 ≠ 0) | (c_9_11 = 0 & csouth
26 = 0 & c_11_13 = 0)) &
27
28     -- 节点15阻塞
29     ((c_13_15 = 1 & c_15_1 ≠ 0) | (c_13_15 = 5 & cwest
30 ≠ 0) | (c_13_15 = 9 & cwest ≠ 0) |
31     (cwest = 1 & c_15_1 ≠ 0) | (c_13_15 = 0 & cwest = 0
32 & c_15_1 = 0)) &
33
34     -- 中心节点17阻塞: 入边有消息但所需出边被占用, 或所有通道为空
35     ((cnorth = 9 & ceast ≠ 0) | (cnorth = 13 & csouth ≠
36 0) |
37     (ceast = 13 & csouth ≠ 0) | (ceast = 1 & cwest ≠
38 0) |
39     (csouth = 5 & cnorth ≠ 0) | (csouth = 1 & cwest ≠
40 0) |
41     (cwest = 5 & cnorth ≠ 0) | (cwest = 9 & ceast ≠ 0)
42 |
43     (cnorth = 0 & ceast = 0 & csouth = 0 & cwest = 0))
44 ))

```

4.2.3 定义各类节点

1. 一进一出可收发, 同3.2.3
2. 二进二出不可收发, 同理使用 `active_channel` 变量来监测当前流向, 保证每次只迭代一条路径
3. 17号节点, 将四条边命名为东西南北, 使用变量 `active_input` 控制哪条边进行操作。
具体实现参见文件 `hard1_strict.smv`

4.3 运行检验

运行 NuSMV -v 2 hard1.smv , 得到如下结果

```
1 size of Y1 = 1.69604e+10 states, 1711 BDD nodes
2 size of Y2 = 3.55957e+10 states, 9403 BDD nodes
3 size of Y3 = 1.09465e+11 states, 24540 BDD nodes
4 size of Y4 = 1.87659e+11 states, 47321 BDD nodes
5 size of Y5 = 3.50516e+11 states, 79916 BDD nodes
6 size of Y6 = 5.58365e+11 states, 125954 BDD nodes
7 size of Y7 = 8.5066e+11 states, 189688 BDD nodes
8 size of Y8 = 1.22328e+12 states, 277690 BDD nodes
9 size of Y9 = 1.6589e+12 states, 388160 BDD nodes
10 size of Y10 = 2.15288e+12 states, 521515 BDD nodes
11 size of Y11 = 2.67687e+12 states, 665662 BDD nodes
12 size of Y12 = 3.20601e+12 states, 810690 BDD nodes
13 size of Y13 = 3.71411e+12 states, 943600 BDD nodes
14 size of Y14 = 4.17698e+12 states, 1049793 BDD nodes
15 size of Y15 = 4.57828e+12 states, 1130450 BDD nodes
16 size of Y16 = 4.91105e+12 states, 1182036 BDD nodes
17 size of Y17 = 5.17566e+12 states, 1209428 BDD nodes
18 size of Y18 = 5.37907e+12 states, 1222228 BDD nodes
19 size of Y19 = 5.5312e+12 states, 1217787 BDD nodes
20 size of Y20 = 5.64284e+12 states, 1198855 BDD nodes
21 size of Y21 = 5.72434e+12 states, 1171639 BDD nodes
22 size of Y22 = 5.78433e+12 states, 1133925 BDD nodes
23 size of Y23 = 5.82932e+12 states, 1084533 BDD nodes
24 size of Y24 = 5.86388e+12 states, 1024732 BDD nodes
25 size of Y25 = 5.8911e+12 states, 953143 BDD nodes
26 size of Y26 = 5.91299e+12 states, 869776 BDD nodes
27 size of Y27 = 5.93073e+12 states, 776483 BDD nodes
28 size of Y28 = 5.94498e+12 states, 678500 BDD nodes
29 size of Y29 = 5.95615e+12 states, 578530 BDD nodes
30 size of Y30 = 5.96462e+12 states, 483723 BDD nodes
31 size of Y31 = 5.97075e+12 states, 400773 BDD nodes
32 size of Y32 = 5.97498e+12 states, 326718 BDD nodes
33 size of Y33 = 5.97775e+12 states, 261139 BDD nodes
34 size of Y34 = 5.97946e+12 states, 205630 BDD nodes
35 size of Y35 = 5.98048e+12 states, 159386 BDD nodes
36 size of Y36 = 5.98106e+12 states, 120937 BDD nodes
37 size of Y37 = 5.98138e+12 states, 90123 BDD nodes
38 size of Y38 = 5.98156e+12 states, 66929 BDD nodes
39 size of Y39 = 5.98165e+12 states, 48831 BDD nodes
40 size of Y40 = 5.9817e+12 states, 35930 BDD nodes
41 size of Y41 = 5.98172e+12 states, 26798 BDD nodes
42 size of Y42 = 5.98173e+12 states, 20473 BDD nodes
43 size of Y43 = 5.98174e+12 states, 16153 BDD nodes
44 size of Y44 = 5.98174e+12 states, 13200 BDD nodes
45 size of Y45 = 5.98174e+12 states, 11761 BDD nodes
46 size of Y46 = 5.98174e+12 states, 10859 BDD nodes
47 size of Y47 = 5.98174e+12 states, 10428 BDD nodes
48 size of Y48 = 5.98174e+12 states, 10329 BDD nodes
49 size of Y49 = 5.98174e+12 states, 10197 BDD nodes
```

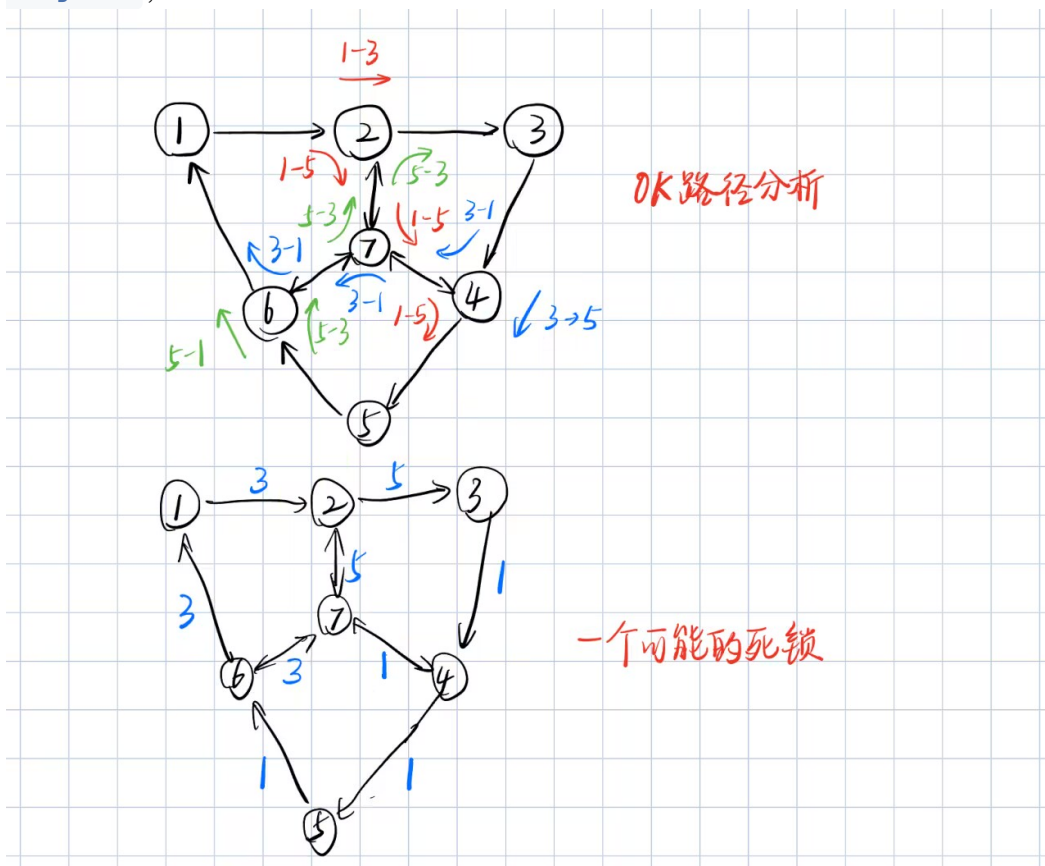

通过向GPT寻求帮助，我了解到NuSMV有一个后续产品nuXmv可以辅助进行并行优化，使用后发现受限于本人的硬件条件，效果仍不佳。

我决定进一步对当前模型进行简化

- 可以明显看到当前模型的节点布局、信道布局、信息流通方向、特定信息流通的最短路径均以中心向四个角的方向对称，因此考虑将4边形简化成同等结构的3边形，这样的简化仍然保留了特定节点的功能，同时在实现逻辑上保留对最短路径的优先选择。少了2个节点和6条边可以大幅减小状态空间。
- 代码参见 [hard1_tri.smv](#)
- 在这次优化后，发现再一次出现了因为假死锁程序终止检验失败的问题，我意识到死锁检验的部分其实已经定义好了什么样的状态是不可行的，假死锁虽然不满足环路条件但是确实也是死锁的一种。所以死锁检验的逻辑是合理的，实际上放宽了状态空间的转移逻辑并不能避免假死锁的到达。

4.6 反思和重新思考

- 回忆最开始的模型图，我删掉了三条不对称的边，因为最短路径都不会经过它们，这是合理的。
- 将假死锁情况扩展回完整的17节点模型，发现问题在于每一条信息都想走中心节点，汇合到一起导致无法前进。如果信息都不走中心节点而是沿着外围前进，会出现类似PPT上的小例子一样的死锁情形。因此我认为这个模型实际上就是存在死锁的。
- 于是我将死锁检验条件直接魔改成了“所有通道全部堵塞”，参见代码 [hard1_tri_wrong.smv](#)，果然运行后发现仍然能得到一个死锁。



- 至此，我认为实际上这个模型存在死锁，我的验证已经成功了。
- 回看发现PPT上死锁定义的配图已经说明了这个结果的必然发生



5 总结与经验

1. 状态空间随着节点数量和信道数量的增加呈现指数级增长，即便是一个简单的模型也会衍生出巨大的状态空间，使得检查变得异常困难。
2. 简化模型能去掉非必要的节点和信道，能指数级降低状态空间大小，提高检查效率。
3. 我曾思考为什么对两个复杂模型都使用了“剪枝”（即简化模型后限制信息从最短路径流通）后，一个模型快速的找到了反例运行成功，而另一个却出现了假死锁，并认为剪枝本来是为了缩小状态空间服务，但是剪枝本质上已经对模型增加了多样的限制，很可能已经破坏了模型本该有的结构，导致出现假死锁（非环路死锁），但事实上，send和process步骤都必须走OK路径，所有这样的限制并不会对模型的检验造成影响。
4. 对于对称的模型，可以利用其对称性质对状态空间进行进一步简化。